# Homework 5
# Cloud SQL Request Logging

**EC528 Cloud Computing | October 28, 2026**

Editable draft for review. The experimental results are verified; the exact billing total remains to be supplied.

Repository: https://github.com/SoymilkWinsAgain/EC528

## 1. System and deployment overview

This homework extends the first application from Homework 4 with PostgreSQL request logging and timing instrumentation. The second application is copied without source changes. Two concurrent clients issue 50,000 requests each, and SQL queries summarize the resulting 100,000-request cohort.

| Resource | Configuration |
|---|---|
| Project | cs528-project-0-508215 (hardcoded in the scripts) |
| Application VM | hw5-server; e2-small; Debian 12; us-east1-b; HTTP port 8080 |
| Cloud SQL | hw5-db; PostgreSQL 16; db-g1-small; us-east1; 10 GB SSD |
| Database and user | hw5 / postgres |
| Existing content | gs://ec528hw2/pages/; 12,000 HTML files; bucket in us-central1 |
| Pub/Sub | hw5-forbidden-requests / hw5-forbidden-requests-sub |
| Runtime identity | hw5-app@cs528-project-0-508215.iam.gserviceaccount.com |

### Request flow

Client -> application 1 -> header extraction and country check -> Cloud Storage or the in-memory file cache -> PostgreSQL transaction -> HTTP response. Requests from banned countries additionally produce a Cloud Logging event and a Pub/Sub message. Application 2 receives, prints and acknowledges the Pub/Sub message.

The server uses ThreadingHTTPServer and a shared two-connection SQLAlchemy pool. All formal-client requests are logged to SQL, including cache hits and rejected requests. Only successful file downloads are cached; the fixed homework content does not need cache invalidation during this run.

### Infrastructure and startup scripts

setup.sh enables APIs, creates the service account and permissions, provisions Pub/Sub, the hourly stopper, Cloud SQL and the VM, and passes source/configuration through VM metadata. startup.sh installs dependencies once using /var/log/startup_already_done, refreshes application files from metadata and runs both applications as systemd services. VM setup does not use SSH commands.

The project roles are roles/cloudsql.client, roles/cloudsql.admin, roles/logging.logWriter, roles/pubsub.publisher and roles/pubsub.subscriber. The service account also receives roles/storage.objectViewer on the content bucket and authenticated invocation access to the function.

## 2. Database schema and request processing

schema.sql defines two tables. The generated request_id is the primary key of request_log and is reused as the primary/foreign key of request_errors. The error table therefore links to the full request without duplicating demographic columns.

| request_log column | PostgreSQL type | Meaning / source |
|---|---|---|
| request_id | BIGINT IDENTITY, PK | Automatically generated request identifier |
| country | TEXT | X-country |
| client_ip | INET | Validated X-client-IP; simulated client metadata |
| gender | TEXT | X-gender |
| age_group | TEXT | X-age; preserve the supplied group label |
| income_group | TEXT | X-income; preserve the supplied group label |
| is_banned | BOOLEAN NOT NULL | Case-insensitive match against the country denylist |
| client_time | TIMESTAMP WITHOUT TIME ZONE | X-time parsed as YYYY-MM-DD HH:MM:SS |
| received_at | TIMESTAMP WITH TIME ZONE | Server receipt time in UTC |
| requested_file | TEXT NOT NULL | Decoded path / requested object name |
| status_code | SMALLINT NOT NULL | HTTP code, constrained to 100-599 |
| run_id | TEXT NOT NULL; default 'manual' | Separates curl, health, trial and formal test records |

| request_errors column | Type / constraint |
|---|---|
| request_id | BIGINT PRIMARY KEY REFERENCES request_log(request_id) |
| received_at | TIMESTAMP WITH TIME ZONE NOT NULL |
| requested_file | TEXT NOT NULL |
| error_code | SMALLINT NOT NULL; 100-599 and not 200 |

The application strips header values, validates the supplied IP address, parses the client timestamp, and preserves age/income as categories. Invalid IP/time values and absent optional headers are recorded as NULL. The banned list includes North Korea, Iran, Cuba, Myanmar, Iraq, Libya, Sudan, Zimbabwe and Syria, plus their two-letter codes.

database.py uses parameterized INSERT statements. It inserts the main row with RETURNING request_id; for a non-200 status it inserts the error row using the same ID. Both inserts commit together, or roll back on failure. Cloud SQL connectivity uses the Python Connector with pg8000, VM service-account credentials and the generated database password.

# 3. Configuration, execution and curl validation

Prerequisites: authenticated Google Cloud CLI with permissions in the hardcoded project, Python 3, and Bash (Git Bash on Windows). The existing ec528hw2 bucket supplies the pages. Run from the repository root:

```
gcloud auth login
# Windows PowerShell with Git Bash
& 'C:\Program Files\Git\bin\bash.exe' hw5/setup.sh
```

The setup script creates missing resources, starts an existing SQL instance and applies the idempotent schema without dropping existing rows. It generates app.env, including a random postgres password. The local file is ignored by Git and is passed privately as VM metadata. Wait for startup completion before making requests.

```
$serverIp = gcloud compute instances describe hw5-server `
    --zone=us-east1-b --project=cs528-project-0-508215 `
    --format='value(networkInterfaces[0].accessConfigs[0].natIP)'
```

The VM uses an ephemeral address, retrieved dynamically. Addresses visible in screenshots are the historical test addresses. In Cloud SQL Studio, select database hw5 and user postgres, using DB_PASSWORD from local app.env. The application already receives its database configuration through startup metadata.

## Three curl requests (assignment part 4)

All three requests carry X-Run-ID: curl-demo. Query request_log with WHERE run_id = 'curl-demo'; query request_errors by joining request_log on request_id and applying the same filter. The displayed empty state refers to this test subset, not every prior row in the database.

| Test | Country / file | Status / ID | Subset counts before -> after |
|---|---|---|---|
| Success | Canada / pages/0.html | 200 / 7 | main: 0 -> 1; errors: 0 -> 0 |
| Missing file | Canada / pages/not-found-hw5.html | 404 / 8 | main: 1 -> 2; errors: 0 -> 1 |
| Banned country | Iran / pages/0.html | 400 / 9 | main: 2 -> 3; errors: 1 -> 2 |

Appendix A includes the actual curl commands/responses and before/after table panels. To avoid duplicate screenshots, the after-state of one request is also the before-state of the next request. The final main-table panel shows all three status codes, and the error panel contains only IDs 8 and 9.

## Load test and analysis commands

Use the updated executable under hw5. Start the following command in each of two terminals, using the same seed and starting the second client without waiting for the first:

```
.\hw5\http-client.exe -d $serverIp -p 8080 -b none -w pages `
    -n 50000 -i 11999 -r 528 -t 30
```

Pause hw5-stop-sql-hourly while SQL is in use. After both clients finish, retain the IDs of the formal requests and export the systemd journal before deleting the VM. Then run analyze_timing.py against the saved local evidence.

```
gcloud compute ssh hw5-server --zone=us-east1-b `
    --project=cs528-project-0-508215 `
    --command="sudo journalctl -u hw5-service1.service -o cat --no-pager"
.\.venv\Scripts\python.exe .\hw5\analyze_timing.py
```

# 4. Concurrent load test and timing results (parts 5-6)

The formal run started October 27, 2026 at 22:57:42 and completed verification October 28 at 01:04:37, America/New_York (UTC-04:00). Both clients used the updated hw5/http-client.exe with n=50000, index=11999, seed=528 and timeout=30 seconds. The server address for this run was 35.237.121.161.

| Run item | Verified result |
|---|---|
| Client 1 | 50,000 requested; exit code 0; 7,562.023 s |
| Client 2 | 50,000 requested; exit code 0; 7,612.136 s |
| Total elapsed including verification | 7,614.903 s (2 h 6 min 54.9 s) |
| Database verification | 100,000 records; 100,000 unique request IDs |
| Timing-log verification | 100,000 matching events; no missing IDs |
| Database / response failures in timing logs | 0 / 0 |

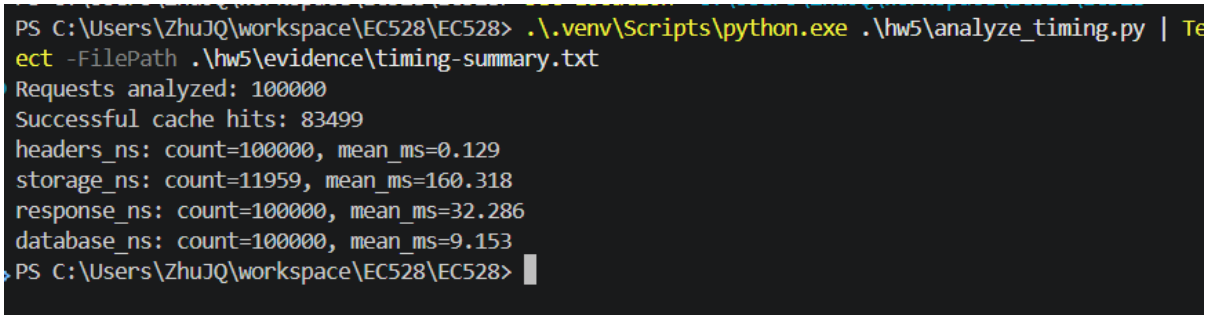
perf_counter_ns is used by the measure context manager. Header extraction, file reading, response writing and database insertion are separate functions. The analyzer matches journal events to stress-request-ids.csv so that trials and unrelated requests do not enter the averages.

| Instrumented operation | Samples | Mean (ms) |
|---|---|---|
| Application request/header extraction | 100,000 | 0.129 |
| Actual Cloud Storage download | 11,959 | 160.318 |
| Server response write and flush | 100,000 | 32.286 |
| Database insertion / transaction | 100,000 | 9.153 |

There were 83,499 successful cache hits. These requests skip the cloud read and have storage_ns=NULL; banned requests also skip storage. Only actual downloads enter the storage average. A small health/check run preceded the formal test. Each cache hit still generates its own SQL row.

Response timing measures the server write/flush operation, not the complete client transfer latency. Pub/Sub, Cloud Logging and other overhead are outside these four measurements. The means use different sample sets and should not be added as an end-to-end request latency.

Figure 1. Output of analyze_timing.py (06-time.png).



# 5. SQL request statistics (part 7)

All statistics use the same fixed formal-run cohort. The country condition excludes unrelated requests that did not carry the simulated client metadata; the ID limits exclude the earlier checks and trials.

```
FROM request_log
WHERE run_id = 'stress-01'
  AND request_id BETWEEN 2146 AND 102159
  AND country IS NOT NULL
```

| Question | Answer |
|---|---|
| Total requests | 100,000 |
| Successful (HTTP 200) | 95,458 |
| Unsuccessful (HTTP other than 200) | 4,542; all returned HTTP 400 |
| Requests from banned countries | 4,542 |
| Male vs Female | Male: 50,232; Female: 49,768 |
| Most frequent age group | 76+; 12,689 requests |
| Most frequent income group | 250k+; 12,625 requests |

## Top five countries

| Rank | Country | Requests |
|---|---|---|
| 1 | Zambia | 562 |
| 2 | Colombia | 561 |
| 3 | Saint Vincent and the Grenadines | 559 |
| 4 | North Korea | 557 |
| 5 | Lithuania | 556 |

COUNT(*) FILTER counts successful, unsuccessful and banned requests. GROUP BY groups gender, country, age_group and income_group; ORDER BY request_count DESC ranks them. LIMIT 5 selects the top countries. The top-country ranking includes rejected requests, so North Korea can appear even though its requests are denied.

The Male and Female totals sum to 100,000. The success and failure totals also sum to 100,000, and the failure count matches the banned-request count for this formal run. Appendix B reproduces the relevant SQL result panels. The full SQL and results remain visible in the original screenshots.

# 6. Hourly stopping, cleanup and cost statement

## Hourly database stopper (part 8)

sql_stopper/main.py is deployed as a private second-generation HTTP Cloud Function, hw5-stop-sql, with entry point stop_database and Python 3.12. Cloud Scheduler invokes it with an OIDC token. The schedule is 0 * * * * in Etc/UTC, once per hour.

The function reads the SQL state and activation policy. When the instance is RUNNABLE with policy ALWAYS, it requests a patch to settings.activationPolicy=NEVER; otherwise it reports that no action is needed. Its runtime service account has roles/cloudsql.admin, and the Scheduler identity has invoker access.

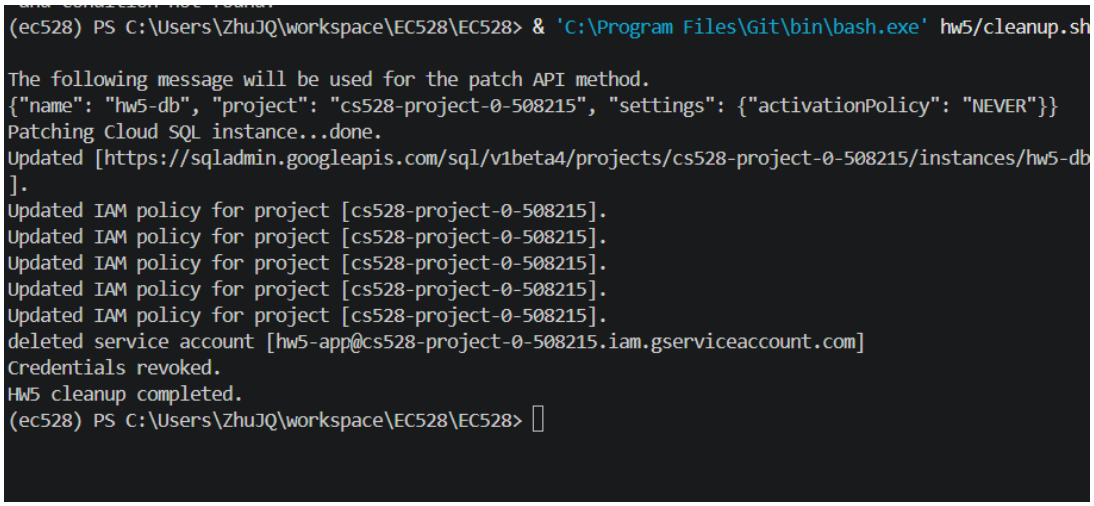
The saved functional test records HTTP 200 and SQL reaching STOPPED/NEVER. The stop operation completed October 27 at 20:00:39 New York time. The schedule was paused during active work and restored afterward. Source evidence: sql-stopper-check.json.

## Cleanup and submission checks

cleanup.sh was executed and verified. It deletes the HW5 VM and boot disk, firewall rule, Pub/Sub subscription/topic, Scheduler job, function and service account, and removes the corresponding IAM bindings. The SQL instance and its data are retained with activationPolicy=NEVER, as required. The existing content bucket and earlier-homework resources are preserved.

Windows compatibility is handled by removing carriage returns from gcloud role output before the IAM-removal loop. Already-removed bucket permissions are skipped. setup.sh, startup.sh and cleanup.sh are recorded as mode 100755, and .gitattributes fixes Bash files to LF line endings. cleanup.sh also attempts ADC revocation.

**Figure 2. Successful final cleanup (08-cleanup.png).**




## Cloud spending - pending verification

The exact monetary total has not yet been verified and is not estimated in this draft. The final version or billing_repord.pdf will report the October 27-28, 2026 project/service cost breakdown, applicable credits and net total, with the billing-report capture time. Any shared-project costs must be identified when interpreting that total.

Billing reference: https://docs.cloud.google.com/billing/docs/how-to/reports

# Appendix A1. HTTP 200: before and after

Panels are cropped to relevant results for readability. Original PNGs are preserved in hw5/evidence and embedded in this Word document. The queries select the curl-demo subset.

**Before 200: request_log empty (01-before-main.png)**

No rows to display

**Before 200: request_errors empty (01-before-errors.png)**

No rows to display

**curl response: HTTP 200 (01-curl-200.png).**




**After 200: request_log contains ID 7 (01-after-main.png).**

| request_id | country | client_ip | gender | age_group | income_group | is_banned | client_time | received_at | requested_ |
|---|---|---|---|---|---|---|---|---|---|
| 7 | Canada | 203.0.113.20 | Female | 26-35 | 40k-60k | false | 2026-10-27... | 2026-10-27T23:04:... | pages/0.htr |

Rows per page: 20 1 – 1 of 1

**After 200: request_errors remains empty (01-after-errors.png).**

No rows to display

# Appendix A2. HTTP 404: before and after

Before this request, the tables are exactly the after-200 state in Appendix A1: one main row and zero error rows. The missing file adds ID 8 to both tables.

**curl response: HTTP 404 (03-curl-404.png).**

```
PS C:\Users\ZhuJQ\workspace\EC528\EC528> curl.exe -sS --max-time 30 -D - -o NUL `
>>     -H "X-country: Canada" `
>>     -H "X-client-IP: 203.0.113.21" `
>>     -H "X-gender: Female" `
>>     -H "X-age: 26-35" `
>>     -H "X-income: 40k-60k" `
>>     -H "X-time: 2026-10-27 12:01:00" `
>>     -H "X-Run-ID: curl-demo" `
>>     "http://34.148.9.114:8080/pages/not-found-hw5.html"
HTTP/1.0 404 Not Found
Server: BaseHTTP/0.6 Python/3.11.2
Date: Tue, 27 Oct 2026 23:08:55 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 15
```

**After 404: request_log contains IDs 7 and 8 (03-main-404.png).**

| request_id | country | client_ip | gender | age_group | income_group | is_banned | client_time | received_at | requested |
|---|---|---|---|---|---|---|---|---|---|
| 7 | Canada | 203.0.113.20 | Female | 26-35 | 40k-60k | false | 2026-10-27... | 2026-10-27T23:04:... | pages/0.ht |
| 8 | Canada | 203.0.113.21 | Female | 26-35 | 40k-60k | false | 2026-10-27... | 2026-10-27T23:08:... | pages/not- |

Rows per page: 20 1 – 2 of 2

**After 404: request_errors contains ID 8 / error 404 (03-errors-404.png).**

| request_id | received_at | requested_file | error_code |
|---|---|---|---|
| 8 | 2026-10-27T23:08:54.637992Z | pages/not-found-hw5.html | 404 |

Rows per page: 20 1 – 1 of 1

# Appendix A3. HTTP 400: before and after

Before this request, the tables are the after-404 state in Appendix A2: two main rows and one error row. The Iran request adds ID 9 with is_banned=true and status 400.

curl response: HTTP 400 (03-curl-400.png).



```
PS C:\Users\ZhuJQ\workspace\EC528\EC528> curl.exe -sS --max-time 30 -D - -o NUL `
>>     -H "X-country: Iran" `
>>     -H "X-client-IP: 203.0.113.22" `
>>     -H "X-gender: Male" `
>>     -H "X-age: 36-45" `
>>     -H "X-income: 60k-100k" `
>>     -H "X-time: 2026-10-27 12:02:00" `
>>     -H "X-Run-ID: curl-demo" `
>>     "http://34.148.9.114:8080/pages/0.html"
HTTP/1.0 400 Bad Request
Server: BaseHTTP/0.6 Python/3.11.2
Date: Tue, 27 Oct 2026 23:10:42 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 18
```

After 400: all three main rows, with status codes 200, 404 and 400 (03-main-400.png).

| request_id | requested_file | country | is_banned | status_code |
|---|---|---|---|---|
| 7 | pages/0.html | Canada | false | 200 |
| 8 | pages/not-found-hw5.html | Canada | false | 404 |
| 9 | pages/0.html | Iran | true | 400 |

Rows per page: 20 1 – 3 of 3

After 400: only the two failures appear in request_errors (03-errors-400.png).

| request_id | received_at | requested_file | error_code |
|---|---|---|---|
| 8 | 2026-10-27T23:08:54.637992Z | pages/not-found-hw5.html | 404 |
| 9 | 2026-10-27T23:10:42.457987Z | pages/0.html | 400 |

Rows per page: 20 1 – 2 of 2

# Appendix B. SQL statistics evidence

These are the recorded result panels from the same cohort selector shown in Section 5. Complete queries and result views are retained in the original screenshots.

**Totals, successful, unsuccessful and banned requests (07-query.png).**

| total_requests | successful | unsuccessful | banned_requests |
|---|---|---|---|
| 100000 | 95458 | 4542 | 4542 |

**Male / Female counts (07-gender.png).**

| gender | request_count |
|---|---|
| Male | 50232 |
| Female | 49768 |

**Top five countries (07-top-countries.png).**

| country | request_count |
|---|---|
| Zambia | 562 |
| Colombia | 561 |
| Saint Vincent and the Grenadines | 559 |
| North Korea | 557 |
| Lithuania | 556 |

**Most frequent age group: 76+ (first result row; 07-ages.png).**

| age_group | request_count |
|---|---|
| 76+ | 12689 |

**Most frequent income group: 250k+ (first result row; 07-income.png).**

| income_group | request_count |
|---|---|
| 250k+ | 12625 |